

Topic 34 — Ollama: Local LLM Runtime Engine

Run large language models on your own machine — free, private, offline.

[New to Python](#)[Intermediate](#)[Expert](#)[Interview Q&A](#)

Layer 1 — New to Python

The one-line idea

Ollama lets you download an AI model onto your own laptop and talk to it — **no internet, no API key, no bill**.

The analogy you already know

You have `node`. You don't *write* the JavaScript engine — you install a runtime, hand it a file, and it executes. Ollama is exactly that, but for AI models. You install the runtime, hand it a model, and it runs.

<code>node app.js</code>	→	<code>ollama run llama3.2</code>
(runtime + your code)		(runtime + a model)

Why bother, when the OpenAI / Anthropic API exists?

Reason	What it means for you
Free	No per-token cost. Experiment all day without watching a billing dashboard.
Private	Data never leaves your machine. Critical for consulting/client work under NDA.
Offline	Works on a plane, or inside a locked-down corporate network.
No rate limits	Hammer it in a loop while testing a RAG pipeline.
Learning	You see the whole stack instead of a black-box HTTPS call.

The honest trade-off

Local models are **smaller and weaker** than GPT-4 / Claude. You use them to *build and test* the pipeline, then swap the model for a hosted one in production. Same code, different endpoint — see the OpenAI-compatibility section in Layer 2.

Getting it running — three commands

```
# 1. Install (macOS / Linux)
curl -fsSL https://ollama.com/install.sh | sh
```

```
# 2. Download a model (~2 GB)
ollama pull llama3.2

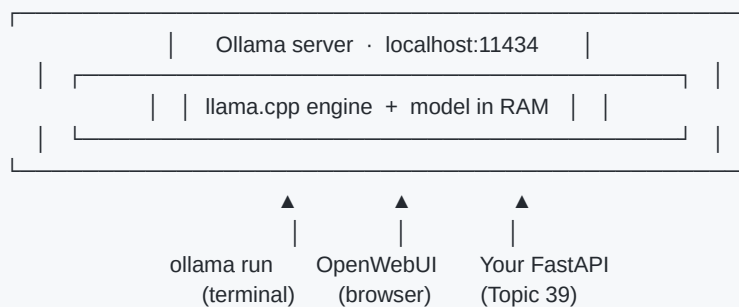
# 3. Chat with it
ollama run llama3.2
>>> Explain a Python decorator in one line.
```

That's it. You now have an LLM running on your machine.

Layer 2 — Intermediate

The mental model: Ollama is a *server*, not a chat app

This is the part beginners miss. When you install Ollama, it starts a **background HTTP server on `localhost:11434`**. The `ollama run` terminal chat is just *one client* talking to that server.



Why this matters

Everything in Topics 35–39 is just **another client** pointed at that same port. Docker, OpenWebUI, FastAPI — all of them are HTTP callers. Once this clicks, the rest of the phase is easy.

The CLI commands worth memorizing

Command	What it does	npm / docker equivalent
<code>ollama pull llama3.2</code>	Download model weights	<code>npm install</code> / <code>docker pull</code>
<code>ollama run llama3.2</code>	Load into RAM + open chat	<code>docker run -it</code>
<code>ollama list</code>	Show models on disk	<code>npm ls</code> / <code>docker images</code>
<code>ollama ps</code>	Show models in RAM right now	<code>docker ps</code>
<code>ollama rm llama3.2</code>	Delete model from disk	<code>docker rmi</code>
<code>ollama serve</code>	Start server manually (foreground)	<code>node server.js</code>
<code>ollama show llama3.2</code>	Inspect params, context size, license	<code>docker inspect</code>

The distinction that trips everyone up

`ollama list` = **on disk**. `ollama ps` = **in memory**.

A model only occupies RAM after its first request, and unloads after 5 minutes idle by default.

Model names and tags

```
ollama pull llama3.2           # defaults to :latest
ollama pull llama3.2:1b        # 1 billion parameters – tiny, fast
ollama pull llama3.2:3b        # 3B – good laptop default
ollama pull qwen2.5-coder:7b    # 7B, code-specialized
ollama pull nomic-embed-text    # embeddings, not chat
```

Identical to Docker tags: `name:tag` . The tag usually encodes **parameter count** (`1b` , `3b` , `7b` , `70b`) and sometimes **quantization** (`q4_K_M`).

Rough RAM budget — the number you actually need when picking a model

Size	RAM needed	Feels like
1–3B	2–4 GB	Fast, dumb. Fine for classification / extraction.
7–8B	6–8 GB	The sweet spot on a 16 GB laptop.
13–14B	10–14 GB	Slow on CPU, fine with a GPU.
70B	40 GB+	Server territory. Not your laptop.

Start with `llama3.2:3b` or `qwen2.5:7b` .

The REST API — what you'll actually use from Python

The CLI is for humans. Your code talks HTTP.

```
# Single-turn completion
curl http://localhost:11434/api/generate -d '{
  "model": "llama3.2",
  "prompt": "Why is the sky blue?",
  "stream": false
}'
```

```
# Multi-turn chat — uses the message roles from your Phase 4 work
curl http://localhost:11434/api/chat -d '{
  "model": "llama3.2",
  "messages": [
    { "role": "system", "content": "You are a terse Python tutor." },
    { "role": "user", "content": "What is a generator?" }
  ],
  "stream": false
}'
```

Connects to Phase 4 (Topics 30–33)

That messages array with role/content is **the ChatML schema from Topic 32**. Ollama takes this JSON and converts it into the model's native instruction format — Alpaca, [INST], or ChatML (Topics 31–33) — using a template baked into the model file. Phase 4 wasn't theory; it's exactly what happens inside this call.

Key endpoints

Endpoint	Purpose
<code>POST /api/generate</code>	Single prompt → completion
<code>POST /api/chat</code>	Message array → reply ← use this one
<code>POST /api/embeddings</code>	Text → vector (for RAG later)
<code>GET /api/tags</code>	List local models
<code>POST /api/pull</code>	Download a model programmatically
<code>POST /v1/chat/completions</code>	OpenAI-compatible — drop-in swap

The strategically important one: OpenAI compatibility

Ollama speaks the OpenAI wire format, so the same client library works against both:

```
from openai import OpenAI

client = OpenAI(
    base_url="http://localhost:11434/v1",  # ← the only real change
    api_key="ollama",                    # ignored, but required
)

resp = client.chat.completions.create(
    model="llama3.2",
    messages=[{"role": "user", "content": "Hello"}],
)

print(resp.choices[0].message.content)
```

✓ The pattern to internalize

Develop locally for free → change two environment variables → ship against OpenAI. Same story for LangChain. Put `base_url` and `api_key` in env vars from day one and the migration is a config change, not a rewrite.

The plain Python version

```
import requests

def ask(prompt: str, model: str = "llama3.2") -> str:
    response = requests.post(
        "http://localhost:11434/api/chat",
        json={
            "model": model,
            "messages": [{"role": "user", "content": prompt}],
            "stream": False,
        },
        timeout=120,
    )
    response.raise_for_status()
    return response.json()["message"]["content"]

print(ask("Give me one Python interview tip."))
```

🔗 Where your earlier topics come back

With `"stream": True`, the response becomes **newline-delimited JSON** — one object per token. That is exactly where **Topic 17 (generators)** and **Topic 18 (async)** get used for real. We wire it properly in Topic 39.

Modelfile — the Dockerfile for models

```
# File: Modelfile
FROM llama3.2

PARAMETER temperature 0.2
PARAMETER num_ctx 8192

SYSTEM """
You are a senior Python reviewer. Reply only with corrected code
and a one-line reason. Never apologise.
"""
```

```
ollama create py-reviewer -f Modelfile
ollama run py-reviewer
```

You've now baked a **persona (Topic 29)** into a reusable named model. The syntax is deliberately Dockerfile-shaped.

Modelfile	Dockerfile
<code>FROM llama3.2</code>	<code>FROM node:20</code>
<code>PARAMETER temperature 0.2</code>	<code>ENV NODE_ENV=production</code>
<code>SYSTEM "..."</code>	<code>CMD [...]</code>
<code>ollama create</code>	<code>docker build</code>

What's actually under the hood

Ollama is a **Go server wrapping** `llama.cpp`, the C++ inference engine. It contributes three things on top:

1. **Model management** — a registry, pull/push, content-addressed layer storage (literally OCI-style layers, like Docker).
2. **Lifecycle management** — loads weights into RAM on first request, keeps them warm, unloads on idle.
3. **A stable HTTP API** — including the OpenAI-compatible shim.

Models ship as **GGUF** files: a single binary containing weights + tokenizer + metadata + the prompt template.

That embedded template is *why* `/api/chat` works — Ollama reads the model's declared instruction format and renders your `messages` array into it.

Quantization — the concept behind the file sizes

Model weights are natively 16-bit floats. **Quantization compresses them to 4 or 5 bits**, trading a little accuracy for a lot of RAM.

Tag	Bits	7B model size	Quality
<code>f16</code>	16	~14 GB	Reference / full precision
<code>q8_0</code>	8	~7 GB	Near-identical
<code>q5_K_M</code>	5	~4.8 GB	Very good
<code>q4_K_M</code>	4	~4.1 GB	Default. Best trade-off.
<code>q3_K_M</code>	3	~3.3 GB	Noticeably degraded

Analogy

Quantization is **image compression for neural networks** — JPEG quality 85 versus a 400 MB PNG. Ollama defaults to `q4_K_M`; unless you have a specific reason, leave it alone.

Environment variables that matter in production

```
OLLAMA_HOST=0.0.0.0:11434    # listen on all interfaces (needed in Docker – Topic 35)
OLLAMA_KEEP_ALIVE=30m        # how long a model stays in RAM (default 5m)
OLLAMA_NUM_PARALLEL=4        # concurrent requests per model
OLLAMA_MAX_LOADED_MODELS=2    # models resident simultaneously
OLLAMA_MODELS=/data/models    # where weights live on disk
```

Two gotchas that will bite you

1. **OLLAMA_HOST=0.0.0.0** — inside a container, the default `127.0.0.1` binding means nothing outside the container can reach it. This is the #1 Docker failure in Topic 35.
2. **OLLAMA_KEEP_ALIVE** — with the 5-minute default, a request after an idle gap pays a multi-second **cold-load penalty** while gigabytes are re-read from disk. That's the difference between a 200 ms response and a 9-second one when benchmarking. Set it to `-1` to keep the model resident during development.

Concurrency reality — where Topic 20 pays off

Inference is **CPU/GPU-bound**, not I/O-bound. One model instance processes requests through a scheduler; `OLLAMA_NUM_PARALLEL` batches them, but every parallel slot carves its own KV-cache out of your RAM.

🔗 **This is the GIL discussion from Topic 20, one layer down**

Your FastAPI handler is async and **I/O-bound** — it's just waiting on an HTTP socket — so `asyncio` is correct on *your* side. But the Ollama process is genuinely **compute-bound**, so throwing 50 concurrent requests at it doesn't make it 50× faster; it queues.

Async on the client, queue on the server. Design for that in Topic 39.

Where Ollama stops — be honest about the ceiling

Limitation	What it means
Not multi-tenant	No auth, no per-user quotas, no rate limiting. Never expose port 11434 to the internet.
No horizontal scaling	One node, one process. Scaling means a load balancer plus N nodes, each holding its own copy of the weights.
Throughput-poor vs vLLM	vLLM's PagedAttention and continuous batching are far ahead for real serving. Ollama optimises for <i>developer ergonomics</i> , not tokens/sec at scale.

🎯 **The correct framing for an interview**

Ollama is the local development and prototyping runtime. It's Docker Compose, not Kubernetes. Saying this unprompted signals you understand production trade-offs rather than just following a tutorial.

🎯 Interview Q&A

Q: What is Ollama, and when would you choose it over a hosted LLM API?

Ollama is a local LLM runtime that wraps `llama.cpp` with model management and an HTTP API on port 11434. Choose it when data privacy prohibits sending content to a third party, when you need zero-cost iteration during development, when you need offline capability, or when you're prototyping a pipeline you'll later point at a hosted model. Choose a hosted API when you need frontier-model quality, real scale, or don't want to manage infrastructure.

Q: You've built a RAG app against Ollama. How much work is it to move to OpenAI in production?

Close to none, if you built it right. Ollama exposes an OpenAI-compatible endpoint at `/v1/chat/completions`, so using the `openai` SDK or LangChain means changing only `base_url` and `api_key` — ideally already environment variables. The real work isn't the swap; it's re-tuning prompts, because a 7B local model and GPT-4 respond differently to identical instructions, and re-checking cost and latency assumptions.

Q: What is quantization, and what does it cost you?

Reducing weight precision from 16-bit floats to 4 or 5 bits, shrinking a 7B model from ~14 GB to ~4 GB. The cost is a small accuracy loss — usually imperceptible at `q4_K_M`, clearly degraded below `q3`. It's what makes local inference possible on consumer hardware at all.

Q: Your local API is fast in a loop, but the first request after a break takes 8 seconds. Why?

Model cold-load. Ollama unloads weights from RAM after `OLLAMA_KEEP_ALIVE` (default 5 minutes) of inactivity, so the next request pays for re-reading gigabytes from disk. Raise `OLLAMA_KEEP_ALIVE`, or send a periodic keep-warm request.

Q: How does Ollama know how to format a prompt for different models?

Each GGUF model file embeds a prompt template declaring its native instruction format — Alpaca, [INST] for LLaMA-2, or ChatML. When you POST a messages array to /api/chat, Ollama renders it through that template into the exact string the model was fine-tuned on. That's why the same JSON works across models with completely different underlying formats.

🔗 MEMORY HOOK

Ollama is node for models.

pull = install · run = execute · list = on disk · ps = in RAM

...and everything real happens over **HTTP on port 11434**.